“CPU Scheduling Simulation System”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINNERING

By

Praneeth Gadila	**2520030257**
Haadi Hasil	**2520030297**
Guru Kiran Matray	**2520030096**

Under the Esteemed Guidance of

Dr. K. Hema
Assistant Professor
Department of Computer Science and Engineering



K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOL

Declaration

The Project Report entitled “CPU Scheduling Simulation System” is a record of bonafide work of Praneeth Gadila-2520030257, Haadi Hasil-2520030297, and Guru Kiran Matray-2520030096, submitted in
partial fulfillment for the award of B. Tech in Computer Science and Engineering to K L (Deemed to be) University. The results embodied in this report describe our implementation and study of CPU scheduling concepts.

Praneeth Gadila	**2520030257**
Haadi Hasil	**2520030297**
Guru Kiran Matray	**2520030096**

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

This is to certify that the mini project report entitled “CPU Scheduling Simulation System” is a bonafide work done and submitted by Praneeth Gadila-2520030257, Haadi Hasil-2520030297, and Guru Kiran Matray-2520030096

in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in Computer Science and Engineering, K L (Deemed to be) University, during the academic year 2026–2027.

Signature of the Guide

Signature of the Course Coordinator **Signature of the HOD**

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, course coordinator, Head of the Department, faculty members, and the university for their guidance, support, and facilities provided during the development of this project.

We thank the faculty of the Department of Computer Science and Engineering for their valuable suggestions and encouragement. Their guidance helped us understand the practical implementation of Operating System concepts.

We are grateful to the Department of Computer Science and Engineering for providing the required academic environment and resources. We also thank our classmates, friends, and families for their support throughout the project.

Finally, we thank everyone who directly or indirectly contributed to the successful completion of the CPU Scheduling Simulation System.

Praneeth Gadila **2520030257**
Haadi Hasil **2520030297**
Guru Kiran Matray **2520030096**

Abstract

CPU Scheduling Simulation System is a Linux-based systems programming project designed to monitor and manage the termination of child processes created by a parent application and provide meaningful termination information to the parent process. In a system where multiple child processes perform different tasks, it is important for the parent process to determine when each child completes its assigned operation and whether the termination occurred normally or abnormally. The proposed system creates multiple child processes using the fork() system call, where each child is assigned a specific service-oriented task. These tasks include data processing, file backup, network service handling, database operations, and security monitoring. Each child performs its respective operation using Linux file handling, process-management, and signal-related APIs before terminating.

The project calculates Waiting Time, Turnaround Time, Response Time, and CPU Utilization, and presents scheduling results through tables and Gantt charts. Round Robin uses a circular ready queue and Linux timer/signal mechanisms, including SIGALRM and setitimer(), to demonstrate time-slice expiration. Separate demonstrations illustrate process creation and synchronization using fork() and waitpid(), anonymous pipe communication, FIFO communication, and signal/timer handling.

Overall, the CPU Scheduling Simulation System provides a practical demonstration of important Operating System and Systems Programming concepts through actual Linux process execution rather than simple message simulation. The individual child services perform real operations such as creating and processing data files, copying and verifying backup files, creating and testing a TCP socket connection, inserting, reading, updating, and verifying database records stored in a file, and scanning the Linux /proc filesystem for active processes. The parent process maintains an event history and provides termination reports and process statistics, including normal terminations, signal-based terminations, process reaping, and zombie handling. Through these operations, the project demonstrates process creation, parent-child relationships, inter-process synchronization, exit status management, signal-based termination, zombie-process prevention, and process lifecycle management. It therefore provides a practical understanding of how Linux creates, executes, monitors, terminates, and cleans up child processes while communicating meaningful termination information to their parent process.

CONTENTS

Chapter / Section	Page (approx.)
Acknowledgement	i
Abstract	ii
Chapter 1 – Introduction	1
Chapter 2 – OS Concepts and Background Study	3
Chapter 3 – System Requirements	6
Chapter 4 – Technology and Tools Used	8
Chapter 5 – System Architecture and Design	10
Chapter 6 – Implementation	13
Chapter 7 – Execution and Demonstration	19
Chapter 8 – Conclusion	23
Chapter 9 – References	25

Page numbers are indicative and may change slightly when the document is opened or printed.

ABSTRACT

The CPU Scheduling Simulation System is a C-based Linux project that demonstrates how an operating system schedules processes competing for CPU time. The application accepts process details—including process ID, arrival time, burst time, and priority—and simulates four algorithms: First Come First Serve (FCFS), non-preemptive Shortest Job First (SJF), non-preemptive Priority Scheduling, and preemptive Round Robin. Process information is maintained through PCB-style structures and dynamic memory allocation.

The simulator computes Waiting Time (WT), Turnaround Time (TAT), Response Time (RT), and CPU Utilization. It displays process results and Gantt charts, including idle intervals and Round Robin time slices. The project also includes Linux demonstrations using fork(), waitpid(), pipe(), FIFO, signals, SIGALRM, and setitimer(). The GUI provides a practical way to enter workloads, select an algorithm, execute the simulation, and inspect outputs.

The project is educational: it models scheduling decisions and related OS mechanisms but does not replace or control the actual Linux kernel scheduler.

CHAPTER 1 – INTRODUCTION

1.1 Background of the Project

A modern operating system manages many processes that may be ready to execute at the same time. On a single CPU core, only one process can execute at a time, so the operating system must choose which ready process receives the CPU. This decision is made by the CPU scheduler. Different scheduling policies produce different execution orders and affect waiting time, response time, turnaround time, and CPU utilization.

The CPU Scheduling Simulation System implements commonly studied scheduling policies in C and provides a graphical interface for observing their behavior. Users enter process attributes and select an algorithm. The simulator determines the execution order, records timing information, calculates performance metrics, and renders a Gantt chart.

1.2 Problem Statement

CPU scheduling algorithms are often learned through manual calculations. This can make it difficult to visualize how arrival times, burst times, priorities, time quantum, and idle periods affect execution. A practical simulator is needed to demonstrate these decisions consistently, produce comparable metrics, and connect scheduling theory with Linux systems programming.

1.3 Scope of the Project

The project covers four CPU scheduling algorithms—FCFS, SJF, Priority, and Round Robin—along with process data management, process-state concepts, Gantt chart generation, and performance analysis. It also provides separate demonstrations of process creation/synchronization, anonymous pipes, FIFO communication, and signal/timer handling. The scheduler is a simulation and does not implement a kernel-level scheduler.

1.4 Objectives of the Project

- Develop a C-based CPU scheduling simulation system on Linux.
- Implement FCFS, non-preemptive SJF, non-preemptive Priority, and preemptive Round Robin.
- Maintain process information using PCB-style structures and dynamic memory.
- Demonstrate process states and lifecycle transitions.
- Use fork() and waitpid() to demonstrate parent-child process management.
- Demonstrate IPC through anonymous pipes and named FIFOs.
- Use signals and setitimer() to demonstrate timer-based time-slice events.
- Calculate WT, TAT, RT, and CPU Utilization.
- Display scheduling results and execution order using Gantt charts.

1.5 Importance of the Application

The application provides an interactive learning environment for comparing scheduling policies. It reduces repetitive manual calculations, makes preemption and idle time visible, and demonstrates how low-level Linux APIs complement high-level scheduling logic.

1.6 Target Users / Audience

The primary audience is students studying Operating Systems, Systems Programming, C programming, and Linux APIs. The simulator can also be used by instructors for classroom demonstrations and by learners reviewing scheduling algorithms.

CHAPTER 2 – OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

The project uses the concepts of process management, CPU scheduling, process states, process control blocks, preemption, ready queues, inter-process communication, signals, timers, and dynamic memory allocation.

• Process: an executing program with its own execution context.
• PCB: a structure holding process attributes such as PID, arrival time, burst time, priority, remaining time, state, and timing values.
• Ready Queue: collection of processes eligible to receive CPU time.
• Preemption: interruption of a running process so another process can execute.
• Context-switching concept: changing the currently scheduled process; modeled in the simulator.
• Process states: NEW, READY, RUNNING, WAITING, and TERMINATED.

A simplified process-state flow is NEW → READY → RUNNING → TERMINATED, with RUNNING → WAITING → READY when a process waits for an event or resource.

2.2 Existing Methods / Algorithms

FCFS schedules processes in arrival order. SJF selects the ready process with the smallest burst time. Priority Scheduling selects the ready process with the highest priority; in this project, a smaller numeric value indicates higher priority. Round Robin assigns each ready process a fixed time quantum and places unfinished processes at the end of a circular ready queue.

FCFS, SJF, and Priority are non-preemptive in this implementation. Round Robin is preemptive and uses timer/signal events to represent time-slice expiration. The algorithms are standard educational policies and are implemented as simulation routines.

2.3 Proposed Approach

The proposed system separates process input, scheduling logic, metrics calculation, Gantt chart recording, and presentation. The user creates a workload through the GUI, chooses an algorithm, and runs the simulation. Each algorithm determines start/completion times and execution segments. The results module computes metrics and the Gantt module records each interval.

The project also provides independent Linux API demonstrations. These examples use actual operating-system process and IPC mechanisms, while the scheduling algorithms operate on simulated process records.

2.4 Advantages of the Proposed Approach

• Allows direct comparison of four scheduling policies using the same workload.
• Visualizes execution order, preemption, and CPU idle intervals.
• Automates standard scheduling metrics.
• Provides a GUI suitable for classroom demonstration.
• Combines scheduling theory with C/Linux systems programming.
• Uses modular source files that are easier to understand and maintain.

CHAPTER 3 – SYSTEM REQUIREMENTS

3.1 Hardware Requirements

Component	Recommended / Minimum
Processor	Any modern dual-core CPU or equivalent
Memory	2 GB RAM minimum; 4 GB or more recommended
Storage	At least 200 MB free space for tools and project
Display	Standard monitor capable of displaying GTK3

	GUI
Input	Keyboard and mouse

3.2 Software Requirements

Software	Purpose
Ubuntu Linux / WSL Ubuntu	Development and execution environment
GCC	C compiler
GTK 3 development libraries	GUI toolkit
pkg-config	Provides GTK compiler/linker flags
GNU Make / build tools	Compilation utilities
Git	Version control and repository management

3.3 Operating System Environment – Ubuntu / WSL

The project is designed for a Linux environment such as Ubuntu. Linux provides the POSIX process, IPC, signal, and timer interfaces used by the demonstrations. The GUI requires GTK 3 and a graphical display environment. On WSL, a supported GUI display configuration is required.

3.4 Development Tools and Compilers

The source code is written in C and compiled using GCC. GTK flags are obtained through pkg-config. The project can be built from the terminal using the commands documented in Chapter 7.

CHAPTER 4 – TECHNOLOGY AND TOOLS USED

4.1 C Programming

C is used for process structures, scheduling algorithms, queue operations, metric calculations, Gantt chart management, and Linux system-call demonstrations. Its direct access to memory and system interfaces makes it suitable for systems programming.

4.2 GCC Compiler

GCC compiles and links the project modules. Warning flags such as -Wall and -Wextra help identify common programming issues during development.

4.3 Linux System Calls and APIs

API / Concept	Purpose in the Project
fork()	Creates a child process in the process-management demonstration.
waitpid()	Synchronizes the parent with a child and collects its exit status.
pipe()	Demonstrates anonymous inter-process communication.
FIFO / mkfifo()	Demonstrates named-pipe communication.
Signals / SIGALRM	Represent asynchronous timer events.
setitimer()	Generates timer events used in the Round Robin demonstration.
malloc() / free()	Allocate and release process, queue, and chart data dynamically.

4.4 Synchronization and IPC

The project demonstrates parent-child synchronization with waitpid(), data transfer through an anonymous pipe, and communication through a named FIFO. These modules illustrate OS communication mechanisms independently from the scheduling simulation.

4.5 Linux Commands and Utilities

The project is compiled and launched from the terminal. Common commands include cd to enter the project directory, ls to inspect files, gcc to compile, and ./scheduler_gui to launch the graphical application.

4.6 Other Tools / Libraries Used

GTK 3 provides the GUI widgets and application window. Cairo is used by the drawing area to render the Gantt chart. Git and GitHub support source-code version control.

CHAPTER 5 – SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

The system is organized into the following layers:

• User Interface (GTK GUI or terminal dashboard)
• Process input and PCB/process data management
• Scheduling algorithms: FCFS, SJF, Priority, Round Robin
• Execution timing and Gantt chart recording
• Metrics calculation and results presentation
• Separate OS demonstrations: fork/waitpid, pipe, FIFO, signals/timer

Workflow: USER INPUT → PROCESS DATA → SELECTED SCHEDULER → SIMULATED CPU EXECUTION → METRICS → GANTT CHART AND RESULTS.

5.2 System Workflow

User enters PID, arrival time, burst time, and optional priority.
Input is validated and stored in process records.
User selects FCFS, SJF, Priority, or Round Robin.
The scheduling module computes execution order and time intervals.
Metrics are calculated from arrival, first-start, burst, and completion times.
The Gantt chart and results are displayed in the GUI.

5.3 Flowchart / Data Flow

Start → Initialize GUI → Add processes → Select algorithm → Validate inputs → Run scheduler → Record execution segments → Calculate metrics → Display table and Gantt chart → Clear or exit.

5.4 Module Description

Module	Responsibility
gui.c	GTK interface, process entry, algorithm selection, result display, Gantt drawing.
process.c / process.h	Process structure, initialization, state and memory management.
fcfs.c	FCFS scheduling and execution intervals.
sjf.c	Non-preemptive SJF scheduling.
priority.c	Non-preemptive priority scheduling.
round_robin.c	Round Robin queue, timer-based slicing, and Gantt entries.
gantt.c / gantt.h	Stores, displays, and frees Gantt chart entries.
process_manager.c	fork()/waitpid() demonstration.
ipc_pipe.c	Anonymous pipe communication demonstration.
fifo.c	Named FIFO communication demonstration.
signals.c	Signal and timer demonstration.
main.c / dashboard.c	Terminal dashboard and menu functions.

5.5 Input and Output Design

Inputs include process ID, arrival time, burst time, priority (for Priority Scheduling), and time quantum (for Round Robin). Outputs include the process execution order, start/completion information, scheduling metrics, and Gantt chart. The GUI also provides controls to add or clear processes and rerun the simulation.

CHAPTER 6 – IMPLEMENTATION

6.1 Module-wise Implementation

The implementation is divided into independent C modules and headers. Shared process and Gantt structures are defined in headers, while algorithm-specific logic is implemented in separate source files. The GUI invokes the selected algorithm and displays its results.

6.2 Algorithm / Pseudocode

FCFS
Sort processes by arrival time. Set current time to the earliest arrival. For each process, if the CPU is idle, advance time to its arrival. Record first start, execute for burst time, record completion, and calculate metrics.

Non-preemptive SJF
At each scheduling decision, select the arrived and unfinished process with the smallest burst time. If no process is ready, advance time to the next arrival. Execute the selected process to completion and repeat.

Non-preemptive Priority
At each decision, select the ready process with the smallest priority number. Resolve ties consistently using arrival/process ordering. Execute the selected process to completion; repeat until all processes finish.

Round Robin
Initialize a ready queue and remaining burst times. Add processes as they arrive. Pop the front process and execute it for min(time quantum, remaining time). Record the slice. If unfinished, enqueue it again. Continue until all processes complete, recording idle intervals when the queue is empty.

6.3 Core C Implementation
The project uses structures to represent processes and Gantt entries. Scheduling modules operate on process arrays, update timing fields, and create execution intervals. Dynamic arrays are allocated and freed through dedicated initialization and cleanup routines.

6.4 Linux System Calls / APIs Used
The process-management module creates a child with fork() and synchronizes using waitpid(). The pipe module transfers process information between parent and child. The FIFO module creates a named pipe and uses open/read/write operations. The signal module installs a SIGALRM handler and configures setitimer() to demonstrate timer expiration.

6.5 Process / State Implementation
The process data model tracks scheduling attributes and state information. The conceptual lifecycle is NEW → READY → RUNNING → TERMINATED. A process may move from RUNNING to WAITING and later return to READY when waiting for an event. Round Robin models preemption by returning unfinished processes to the ready queue.

6.6 IPC / Synchronization Implementation
The anonymous pipe demonstration uses a parent-child relationship for data transfer. The FIFO demonstration uses a named pipe path for communication. waitpid() ensures the parent collects the child's completion status in the process-management demonstration.

6.7 Memory Management Implementation
Dynamic memory is used for process records, ready-queue storage, and Gantt chart entries. Allocation is checked before use, and cleanup functions release allocated memory to avoid leaks.

6.8 User Interface Implementation
The GTK3 GUI provides process input fields, an algorithm selector, an Add Process action, a Clear action, a results area, and a Gantt drawing area. Priority input is relevant to Priority Scheduling, while the time-quantum field is used for Round Robin. Cairo draws the Gantt chart from recorded execution intervals.

CHAPTER 7 – EXECUTION AND DEMONSTRATION

7.1 Compilation in Ubuntu
Install the compiler and GTK development dependencies:

```
sudo apt update
sudo apt install build-essential libgtk-3-dev
```

Navigate to the project directory:

```
cd ~/cpu_scheduler
```

Compile the GUI:
gcc -Wall -Wextra -g gui.c process.c fcfs.c sjf.c priority.c round_robin.c gantt.c -o scheduler_gui $(pkg-config --cflags --libs gtk+-3.0)

7.2 Execution Commands

Launch the GUI with:
./scheduler_gui
The terminal dashboard can be compiled and run separately:
gcc -Wall -Wextra -g main.c process.c fcfs.c sjf.c priority.c round_robin.c process_manager.c ipc_pipe.c fifo.c signals.c gantt.c dashboard.c -o scheduler
./scheduler

7.3 Sample Inputs and Outputs

A sample workload for demonstration is shown below. The same process set can be run under different algorithms to compare execution order and metrics.

Process	Arrival Time	Burst Time	Priority
P1	0	5	2
P2	1	3	1
P3	2	4	3

For Round Robin, choose a time quantum of 2. The resulting Gantt sequence depends on the arrival times and ready-queue order; the GUI displays the computed segments and metrics.

7.4 Screenshots of Execution

Insert the project screenshots in this section: (1) main GUI with process input, (2) selected scheduling algorithm, (3) scheduling results and metrics, and (4) Gantt chart. Captions should identify the algorithm and workload used.

7.5 Demonstration of OS Concepts

- Process creation and parent-child synchronization using fork() and waitpid().
- Anonymous pipe data transfer between related processes.
- Named FIFO communication.
- Signal handling and timer event generation using SIGALRM and setitimer().
- Dynamic allocation and cleanup of process and Gantt data.

CHAPTER 8 – CONCLUSION

8.1 Summary of the Project

The CPU Scheduling Simulation System provides a practical implementation of four CPU scheduling algorithms in C on Linux. It accepts process attributes, simulates scheduling, calculates standard metrics, and visualizes execution through Gantt charts and a graphical interface. Separate demonstrations connect the project with Linux process and IPC APIs.

8.2 Objectives Achieved

The project implements FCFS, SJF, Priority, and Round Robin; maintains process data; demonstrates process-state concepts; uses dynamic memory; provides IPC and signal/timer demonstrations; calculates WT, TAT, RT, and CPU Utilization; and displays results through a GUI and Gantt chart.

8.3 Results Achieved

The application runs the selected scheduling routine on the entered workload and presents its execution sequence and computed metrics. The Gantt chart shows process intervals and idle time where applicable. Round Robin records individual time slices to make preemption visible. Results are workload-dependent and are generated at runtime rather than asserted as fixed benchmark values.

8.4 Skills and Learning Outcomes

- Understanding and implementing CPU scheduling algorithms.
- Applying C structures, arrays, dynamic allocation, and modular design.
- Using Linux process creation and synchronization APIs.
- Understanding pipes, FIFOs, signals, and timer events.

• Calculating scheduling metrics and interpreting Gantt charts.
• Building a GTK-based GUI and compiling a multi-file C project.
In conclusion, the project bridges theoretical Operating Systems concepts with practical C/Linux implementation. It is a simulator for learning and analysis, not a modification of the Linux kernel scheduler.

CHAPTER 9 – REFERENCES

9.1 Books

Abraham Silberschatz, Peter Baer Galvin, and Greg Gagne, Operating System Concepts, Wiley.
Andrew S. Tanenbaum and Herbert Bos, Modern Operating Systems, Pearson.
William Stallings, Operating Systems: Internals and Design Principles, Pearson.
Brian W. Kernighan and Dennis M. Ritchie, The C Programming Language, Prentice Hall.

9.2 Linux / GNU Documentation

GNU Compiler Collection (GCC) documentation.
GTK 3 Reference Documentation.
Linux man-pages: fork(2), waitpid(2), pipe(2), mkfifo(3), signal(7), setitimer(2).
GNU C Library documentation.

9.3 Online Resources

Ubuntu Documentation – Linux environment, package installation, and development tools.
Linux man-pages Project – system-call and library-function references.
GTK Project – GTK 3 API and application development documentation.
GitHub repository: https://github.com/pran12171/CORE_OS_proj